Quantstamp

NilDB

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Infra
Timeline	2025-10-29 through 2025-10-29
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• https://github.com/NillionNetwork/nildb #c59b380 • NillionNetwork/nuc-ts #ba49838 • NillionNetwork/nildb-infra #bb723e3
Auditors	• Albert Heinle Auditing Engineer

Documentation quality	Undetermined
Test quality	Undetermined
Total Findings	23 Fixed: 4 Acknowledged: 19
High severity findings	8 Fixed: 4 Acknowledged: 4
Medium severity findings	11 Acknowledged: 11
Low severity findings	2 Acknowledged: 2
Undetermined severity findings	0
Informational findings	2 Acknowledged: 2

Summary of Findings

The audit of `nildb` and `nuc-ts` and its associated infrastructure identified findings encompassing the application, containerization, and cloud environment layers. Overall, the system demonstrates a mature engineering foundation with adherence to modern DevSecOps practices. Additionally, the team has demonstrated a great attention to detail for input validation. There are some gaps, however, if unaddressed, could expose the environment to operational and security risks.

At the application layer, the most significant issues relate to token management and input validation. We observed that NUC tokens can be created without an explicit expiration, and that signature verification is decoupled from token parsing. This is a pattern that could allow misuse or accidental acceptance of invalid tokens. Additionally, a lack of enforced immutability in critical objects weaken the application's integrity guarantees. Improving validation rigor, enforcing immutability, and strengthening exception handling mechanisms would meaningfully reduce the attack surface.

Within the container and CI/CD pipeline, several findings relate to configuration hardening and supply chain integrity. Docker containers are running as root, and not all include proper health checks. Secrets were identified directly in ECS task definitions, while GitHub Actions workflows lacked permission scoping and pinning to specific commit SHAs. This is introducing potential risks of unauthorized access or dependency hijacking. Furthermore, code coverage thresholds are not enforced in the CI/CD process, albeit being currently in a very healthy state.

In the cloud infrastructure, AWS-related issues represent both configuration and resilience gaps. Several AWS accounts were found without enforced multi-factor authentication (MFA), and production RDS instances were missing deletion protection. Additionally, ECR repositories lack tag immutability, and vulnerability scanning of ECR images is incomplete. ECS task definitions also showed insufficient network isolation, which could expose services to unintended access.

Other notable concerns include divergence between test and production configurations, and lack of consistent timeout settings in GitHub Actions jobs. Together, these suggest that while the system benefits from automated processes, certain operational safeguards and quality gates remain incomplete.

Overall, addressing the identified issues will enhance the platform's security posture and operational resilience.

ID	DESCRIPTION	SEVERITY	STATUS
NIL-1	Nuc Tokens Can Be Built without Explicit Expiration	• High	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
NIL-2	Aws Accounts Missing Multi-Factor Authentication (Mfa)	High	Acknowledged
NIL-3	Ncu Token Parsing and Signature Verification Are Decoupled, Allowing Potential Misuse	High	Fixed
NIL-4	Root User for Docker Containers	High	Fixed
NIL-5	Ecr Image Vulnerability Scanning	High	Acknowledged
NIL-6	Ecs Task Network Isolation	High	Acknowledged
NIL-7	Github Actions Permissions Restriction	High	Fixed
NIL-8	Secrets Embedded in Ecs Task Definition Container Definitions	High	Acknowledged
NIL-9	Ecr Image Tag Immutability	Medium	Acknowledged
NIL-10	Ecr Repository Encryption with Custom Key	Medium	Acknowledged
NIL-11	Create Volume for System Logs	Medium	Acknowledged
NIL-12	Pin Github Actions to Commit Sha	Medium	Acknowledged
NIL-13	Docker Container Health Check	Medium	Acknowledged
NIL-14	Default Response for Api Endpoints	Medium	Acknowledged
NIL-15	Request Body Definition for Mutation Operations	Medium	Acknowledged
NIL-16	Immutable Objects Not Enforced	Medium	Acknowledged
NIL-17	Test Environment Services Not Aligned with Production Configurations	Medium	Acknowledged
NIL-18	Aws Production Rds Instances Did Not Have Deletion Protection	Medium	Acknowledged
NIL-19	Ensure Language Specific Boolean Interpretations of Types	Medium	Acknowledged
NIL-20	Code Coverage Threshold Not Enforced in Ci/cd Pipeline	Low	Acknowledged
NIL-21	Overly Permissive Regular Expression for Did Validation	Low	Acknowledged
NIL-22	Github Actions Job Timeout	Informational	Acknowledged
NIL-23	Lack of Exception Handling for Potentially Throwing Functions	Informational	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/NillionNetwork/nildb`

Included Paths: `packages/api/src`

Repo: `https://github.com/NillionNetwork/nildb`

Included Paths: `src`

Repo: `https://github.com/NillionNetwork/nildb`

Included Paths: `src`

Findings

NIL-1 Nuc Tokens Can Be Built without Explicit Expiration

High | Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `18898a873c893afb00aa649ff52b827514174d98`.
The client provided the following explanation:

> -

Update

Marked as "Fixed" by the client.
Addressed in: `18898a873c893afb00aa649ff52b827514174d98`.
The client provided the following explanation:

```
-
```

Description: `nuc-ts-main/src/nuc/builder.ts` : In `builder.ts` , the NUC token construction logic allows tokens to be generated without specifying an expiration (`exp`) claim or equivalent time-to-live property. This means that tokens can remain valid indefinitely unless explicitly revoked.

Tokens without expiration pose a significant security risk: if such a token is ever leaked, it can be reused indefinitely to authenticate or authorize malicious activity. This design also makes it difficult to enforce session lifetimes, comply with best practices (such as short-lived credentials), or invalidate old tokens automatically.

Industry-standard token systems (e.g., JWT, OAuth2 access tokens) always enforce finite validity periods to limit exposure in the event of credential compromise. The absence of an enforced expiration undermines the security model of the authentication layer.

Recommendation: Require all tokens generated via builder.ts to include an explicit expiration time, similar to the enforcement of

```
if (!this._audience || !this._subject || !this._command) {
      throw new Error("Audience, subject, and command are required fields.");
    }
```

in the same file.

Furthermore, enforce a maximum expiration, so that there is a cap on the validity of any issued NUC token.

NIL-2
Aws Accounts Missing Multi-Factor Authentication (Mfa)

High | Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> As discussed in the Telegram group chat, the IAM accounts found without MFA are the 'Jenkins' and 'Deployer' users; these are two "bot" accounts, used for automated infrastructure
> deployment of some of our infrastructure which is being phased out. They don't have anything to do with nilDB or its deployment. They cannot have MFA protections in place as
> they aren't human users who could be expected to verify MFA challenges. Further to this, they are in our management account where we don't have nilDB
> workloads running.

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> As discussed in the Telegram group chat, the IAM accounts found without MFA are the 'Jenkins' and 'Deployer' users; these are two "bot" accounts, used for automated infrastructure
> deployment of some of our infrastructure which is being phased out. They don't have anything to do with nilDB or its deployment. They cannot have MFA protections in place as
> they aren't human users who could be expected to verify MFA challenges. Further to this, they are in our management account where we don't have nilDB
> workloads running.

Description: Several AWS accounts associated with the organization do not have Multi-Factor Authentication (MFA) enabled for privileged users or the root account. Without MFA, these accounts rely solely on passwords, which can be compromised through phishing, credential leaks, or brute-force attacks. This significantly increases the risk of unauthorized access, potential data exfiltration, and disruption of critical infrastructure.

MFA is a fundamental security control recommended by AWS and industry standards (e.g., CIS AWS Foundations Benchmark) to protect against account compromise, especially for accounts with administrative privileges. The absence of MFA represents a critical gap in the organization's cloud security posture.

Recommendation: Enable MFA on AWS console accounts.

NIL-3
Ncu Token Parsing and Signature Verification Are Decoupled, Allowing Potential Misuse

• High ⓘ Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `9dc205b214693e1aafcf52488cae0374c0f88bd9`.
The client provided the following explanation:

> Will address in due course

Update
Marked as "Fixed" by the client.
Addressed in: `9dc205b214693e1aafcf52488cae0374c0f88bd9`.
The client provided the following explanation:

> Will address in due course

Description: The library allows NUC tokens, which serve as authentication/authorization tokens, to be parsed without enforcing signature validation and expiry check. This separation of concerns between parsing and signature verification introduces the risk that developers may handle token parsing without performing the required validation step. If a developer omits the explicit call to validate the token signature, an attacker could supply forged or modified tokens that would still be accepted at the parsing stage, potentially resulting in unauthorized access or privilege escalation.

This design pattern contrasts with common secure token handling implementations, such as JSON Web Tokens (JWTs), where signature verification is performed by default during parsing. Allowing unvalidated tokens to be parsed or partially processed before verification increases the attack surface and encourages insecure usage patterns.

Recommendation: Integrate signature verification directly into the token parsing process so that tokens are validated by default. Parsing without verification should only be possible through an explicit, documented opt-out mechanism intended strictly for debugging or development purposes. Additionally, update the API documentation to clearly warn about the risks of handling unvalidated tokens and to highlight the secure default behavior.

NIL-4 Root User for Docker Containers

• High ⓘ Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `1f0c78cd68d195f28869d4efdc592cc8d1e88045`.
The client provided the following explanation:

> Will address in due course

Update
Marked as "Fixed" by the client.
Addressed in: `1f0c78cd68d195f28869d4efdc592cc8d1e88045`.
The client provided the following explanation:

> Will address in due course

Description: When creating a Docker container, it is possible to set the user who is actually running the application and any command on the container. It is important to specifically use the `USER` directive in any Dockerfile to ensure that the user is not root and has unnecessary privileges.

Affected Files:
- `./Dockerfile`

Recommendation: Have at least one `USER` directive in your Dockerfile, and the last user directive should not reference the root user or root group.

NIL-5 Ecr Image Vulnerability Scanning

• High ⓘ Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> None of the affected ECR repositories listed pertain to nilDB in any way. This does not impact the scope of the audited products at all.

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> None of the affected ECR repositories listed pertain to nilDB in any way. This does not impact the scope of the audited products at all.

Description: The ECR container registry should do a vulnerability scan on every pushed image.

Affected Resources:

- `bridge-ui`
- `builder-cross`
- `builder-rust`
- `datablocks-api`
- `deployer`
- `docker-002F-deployer`
- `faucet`
- `nilchaind-builder`
- `nillion-functional-tests`
- `nillion-load-tool`
- `nillion-node`
- `nillion-node-grpc`
- `nillion-report-generator`
- `nilliond-builder`
- `optimism-ci-builder`
- `retailtoken-accuser`
- `retailtoken-task-manager`
- `serverless-serverless-codeparty-testnet-fe`
- `serverless-serverless-codeparty-testnet-nucleus`
- `serverless-serverless-faucet-captcha-validator-testnet-1`
- `serverless-sls-fe-tester-testnet-photon`
- `test`
- `zk-circuits-ci-builder`

Recommendation: For every resource of type `aws_ecr_repository`, ensure that the `image_scanning_configuration` block exists and has its key `scan_on_push` set to `true`.

NIL-6 Ecs Task Network Isolation

High | Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> None of the affected ECS tasks listed pertain to nilDB in any way. This does not impact the scope of the audited products at all.

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> None of the affected ECS tasks listed pertain to nilDB in any way. This does not impact the scope of the audited products at all.

Description: Containers should be isolated as much as possible from the host system. This includes networking, and a ECS task should never be set to use the hosts network directly.

Affected Resources:

- `monitoring-prod-monitoring`
- `monitoring-prod-prometheusrw`
- `monitoring-staging-monitoring`
- `monitoring-staging-prometheusrw`

Recommendation: For every resource of type `aws_ecs_task_definition`, ensure that the `network_mode` is not set to `host`.

NIL-7 Github Actions Permissions Restriction

High Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `bda47b97dd281c78eade92b3c767c7a3e7e0e191`.
The client provided the following explanation:

> -

Update

Marked as "Fixed" by the client.
Addressed in: `bda47b97dd281c78eade92b3c767c7a3e7e0e191`.
The client provided the following explanation:

> -

Description: The default GitHub token used for GitHub actions has very wide permissions. These should be limited depending on the specific workflow.

Affected Files:

- `./.github/workflows/ci.yaml`

Recommendation: For every workflow, ensure that the `permission` section exists and only the necessary permissions are set.

NIL-8 Secrets Embedded in Ecs Task Definition Container Definitions

High Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> As discussed in the Telegram chat, the Task Definitions that were found with plaintext secrets exposed in the environment are for workloads in our management account and have nothing to do with our running nilDB deployments. They are part of the legacy infrastructure which is being phased out. All nilDB workloads running in our other accounts us AWS SecretsManager with `valueFrom` composition to ensure secrets aren't exposed with plaintext anywhere.

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> As discussed in the Telegram chat, the Task Definitions that were found with plaintext secrets exposed in the environment are for workloads in our management account and have nothing to do with our running nilDB deployments. They are part of the legacy infrastructure which is being phased out. All nilDB workloads running in our other accounts us AWS SecretsManager with `valueFrom` composition to ensure secrets aren't exposed with plaintext anywhere.

Description: The ECS task definitions include sensitive values directly within the `container_definitions` block instead of referencing them through the dedicated secrets section. This practice embeds credentials or API keys in plain text within the task definition JSON, making them visible in Terraform state files, CloudFormation templates, version control systems, or the AWS console. Because ECS task definitions are often versioned and retrievable via the AWS API, any user with read permissions on ECS task definitions could inadvertently gain access to these secrets. This exposure risk increases the likelihood of unauthorized access to dependent systems or data breaches.

Recommendation: Refactor ECS task definitions to use the secrets parameter under each container definition. Store sensitive values securely in AWS Secrets Manager or SSM Parameter Store, and reference them using the valueFrom field. Also, review existing task definitions and Terraform state files to identify and rotate any exposed credentials. Ensure code review and CI/CD checks enforce the use of secret references instead of hardcoded values.

NIL-9 Ecr Image Tag Immutability

• Medium ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> None of the affected ECR repositories listed pertain to nilDB in any way. This does not impact the scope of the audited products at all.

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> None of the affected ECR repositories listed pertain to nilDB in any way. This does not impact the scope of the audited products at all.

Description: In order to maintain stability with respect to image tags and versions, it is important to set the tags in ECR to be immutable.

Affected Resources:

- `bridge-ui`
- `builder-cross`
- `builder-rust`
- `datablocks-api`
- `deployer`
- `docker-002F-deployer`
- `faucet`
- `nilchaind-builder`
- `nillion-functional-tests`
- `nillion-load-tool`
- `nillion-node`
- `nillion-node-grpc`
- `nillion-report-generator`
- `nilliond-builder`
- `optimism-ci-builder`
- `retailtoken-accuser`
- `retailtoken-task-manager`
- `serverless-serverless-codeparty-testnet-fe`
- `serverless-serverless-codeparty-testnet-nucleus`
- `serverless-serverless-faucet-captcha-validator-testnet-1`
- `serverless-sls-fe-tester-testnet-photon`
- `test`
- `zk-circuits-ci-builder`

Recommendation: For every resource of type `aws_ecr_repository`, ensure that the `image_tag_mutability` is set to `IMMUTABLE` (default is `MUTABLE`).

NIL-10 Ecr Repository Encryption with Custom Key

• Medium ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> We accept the default AWS encryption key as satisfactory. Container image hosting is being moved to GHCR

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> We accept the default AWS encryption key as satisfactory. Container image hosting is being moved to GHCR

Description: Using custom keys instead of the default AWS managed key is always to be preferred, as one has better control over the key. This check fails if no custom key is used for the encryption of the ECR repository.

Recommendation: For every resource of type `aws_ecr_repository`, ensure that the `encryption_configuration` block exists, and has a `kms_key` defined.

NIL-11 Create Volume for System Logs

• Medium Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> We ship logs to decentralised storage infrastructure. Log persistence in containers is not of concern.

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> We ship logs to decentralised storage infrastructure. Log persistence in containers is not of concern.

Description: In linux systems, important operating system logs are stored in the `/var/log` subfolder. This folder should always be made available to the host through a volume, so that log tracking and log analysis systems can capture them.

Affected Files:
- `./Dockerfile`

Recommendation: In every Dockerfile, there should be a VOLUME directive which has `/var/log` as an argument.

NIL-12 Pin Github Actions to Commit Sha

• Medium Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Description: When using external actions in your workflow, it is important to ensure that the version remains the same to avoid run-failures.

Affected Files:
- `./.github/workflows/cd-image.yaml`
- `./.github/workflows/ci.yaml`

Recommendation: For every `uses` instruction in your GitHub actions, ensure that it is referenced by its SHA hash, and not just the version tag.

NIL-13 Docker Container Health Check

• Medium Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> nilDB has built in liveness and readiness endpoints which is what our infrastructure relies on.

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> nilDB has built in liveness and readiness endpoints which is what our infrastructure relies on.

Description: Dockerfiles have an instruction called `HEALTHCHECK`. It enables a user to define a command to figure out if the program(s) running inside the container are working properly. It is generally advisable to have healthchecks in place to assist monitoring of running containers.

Affected Files:
- `./Dockerfile`

Recommendation: Have at least one `HEALTHCHECK` instruction in your Dockerfile.

NIL-14 Default Response for Api Endpoints

Medium | Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> This is handled in https://github.com/NillionNetwork/nildb/blob/main/packages/api/src/common/handler.ts#L52
> Internal errors are mapped to a finite set of error codes and human readable messages before being returned to the user. We can probably move to a system were verbose errors are only returned in our dev environments but not production.

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> This is handled in https://github.com/NillionNetwork/nildb/blob/main/packages/api/src/common/handler.ts#L52
> Internal errors are mapped to a finite set of error codes and human readable messages before being returned to the user. We can probably move to a system were verbose errors are only returned in our dev environments but not production.

Description: In order to avoid a too detailed error response to the user, one should always create common error and a default response for every endpoint.

Affected Resources:
- Exported OpenAPI specification JSON from the given project.

Recommendation: Every endpoint needs to have default responses for common response codes ready to avoid forwarding detailed error messages. This should include at least 401, 403, 404, 429 and 500 error codes, and `default`.

NIL-15 Request Body Definition for Mutation Operations

Medium | Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Unclear what the auditors mean here. All endpoints explicitly declare the http method they support and every endpoint accepting input uses zod for validation.

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Unclear what the auditors mean here. All endpoints explicitly declare the http method they support and every endpoint accepting input uses zod for validation.

Description: The `requestBody` for operations such as `POST`, `PUT`, and `PATCH` should be defined, as it describes and limits the specific input types to the API. Its absence is an indicator that the endpoint accepts any input and is hence vulnerable to request forgery attacks.

Affected Resources:

- Exported OpenAPI specification JSON from the given project.

Recommendation: To fix this, ensure that the `requestBody` key is included in every `POST`, `PUT` or `PATCH` operation.

NIL-16 Immutable Objects Not Enforced

Medium · Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Will address in due course

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Will address in due course

Description: In src/core/signer.ts, several function exports and configuration objects could benefit from immutability enforcement using Object.freeze(). Currently, these objects remain mutable at runtime, allowing accidental or malicious modifications to function properties or configuration constants after module initialization.

While this may not immediately impact functionality, it increases the risk of unintended side effects and inconsistent behavior if downstream code mutates shared objects. In security-sensitive components such as signer.ts, which handles cryptographic signing operations, ensuring immutability improves predictability and guards against subtle tampering bugs.

Recommendation: Apply Object.freeze() to objects and function exports that are not meant to change during execution. This is not limited to `signer.ts`, but the entire code-base should be examined for this.

NIL-17 Test Environment Services Not Aligned with Production Configurations

Medium · Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Will address in due course

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Will address in due course

Description: The repository's test setup uses docker-compose to spin up dependent services such as MongoDB and PostgreSQL. However, the configurations for these services (e.g., version, authentication settings, resource limits, and storage parameters) differ from those used in production.

This discrepancy reduces the reliability of integration tests. Certain issues, such as schema incompatibilities, query planner differences, authentication behavior, or version-specific bugs, may go undetected in the test environment but surface in production. Mismatched service

configurations can also invalidate performance assumptions or data integrity guarantees.

Recommendation: Align the test environment configurations with the production setup as closely as possible:

- Use the same versions of MongoDB and PostgreSQL as in production.
- Replicate key configuration parameters.

Where possible, codify these configurations in the docker-compose.yml file or in a .env.test file that mirrors production defaults. Document any intentional deviations (e.g., for local testing efficiency) and periodically validate that test and production environments remain compatible.

NIL-18
Aws Production Rds Instances Did Not Have Deletion Protection

Medium | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Will address in due course

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Will address in due course

Description: The production Amazon RDS instances do not have deletion protection enabled. This setting prevents accidental or unauthorized deletion of database instances, which is a critical safeguard for preserving production data integrity.

Without deletion protection, an administrative action, misconfigured infrastructure-as-code deployment, or malicious actor with sufficient privileges could permanently delete the database instance. Even if backups exist, restoring a large production database can lead to significant downtime, data loss, and operational disruption.

This issue represents a violation of AWS best practices for data durability and business continuity, especially for production workloads.

Recommendation: Enable deletion protection on all production RDS instances. This can be done via the AWS Management Console, AWS CLI, or Terraform.

NIL-19
Ensure Language Specific Boolean Interpretations of Types

Medium | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Will address in due course

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Will address in due course

Description: In the code, 0 is marked as `NULL` as well for expiration, which would mean that a `exp: 0` field, which means immediately expired in normal cases, e.g. for debugging and testing, would be interpreted as "never expire" as per the UCAN specification. This is due to the code lines in `src/validator/temporal.ts`:

```
export function validateTemporalProperties(
  payload: Payload,
  currentTime: number,
): void {
```

```
  // Convert currentTime from milliseconds to seconds for comparison
  const currentTimeInSeconds = Math.floor(currentTime / 1000);

  if (payload.exp && payload.exp <= currentTimeInSeconds) {
    Log.debug(
      { expiredAt: payload.exp, now: currentTimeInSeconds },
      TOKEN_EXPIRED,
    );
    throw new Error(TOKEN_EXPIRED);
  }

```

Recommendation: Review the entire code-base for `null` vs `0` boolean JavaScript handling.

NIL-20
Code Coverage Threshold Not Enforced in Ci/cd Pipeline

• Low ⓘ Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Will address in due course

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Will address in due course

Description: The continuous integration pipeline currently collects code coverage metrics using Vitest but does not enforce a minimum coverage threshold. This means that even if future code changes significantly reduce test coverage, the pipeline will still report success and allow merges or deployments to proceed.

While the current coverage level is commendably high (above 80%), the lack of automated enforcement creates a risk of gradual test erosion over time. Without a hard threshold, developers may inadvertently commit code without sufficient test coverage, leading to reduced test confidence and potential undetected regressions.

Recommendation: Configure Vitest to enforce a minimum coverage threshold that aligns with the project's quality goals. This can be achieved by updating the Vitest configuration (e.g., vitest.config.ts) to include a coverage threshold section:

```
export default defineConfig({
  test: {
    coverage: {
      lines: 80,
      functions: 80,
      branches: 80,
      statements: 80,
      thresholds: true,
    },
  },
});
```

Integrate this enforcement step into the CI/CD job so that builds fail if coverage drops below the defined standard. Periodically review the thresholds to ensure they reflect the evolving complexity and risk profile of the codebase.

NIL-21 Overly Permissive Regular Expression for Did Validation

• Low ⓘ Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:

> Will address in due course

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Description: In `nuc-ts-main/src/core/did/did.ts` at line 124, the `DID_EXPRESSION` regular expression used for parsing and validating decentralized identifiers (DIDs) is overly broad. The current expression allows any string after the first colon, meaning malformed or non-standard DIDs could be incorrectly accepted.

Specifically, the second portion after the did: prefix is not constrained to recognized DID methods. While only key, ethr, and nil are supported in the current implementation, the existing regex does not enforce this, leaving room for inconsistencies or potentially malicious input that superficially matches the pattern but doesn't conform to expected DID formats.

This lack of strict validation can result in invalid identifiers being parsed or stored, leading to subtle bugs, mismatched key handling, or interoperability issues when interacting with other systems expecting standard DID method prefixes.

Recommendation: Tighten the regular expression to explicitly enumerate supported DID methods. For example:

```
const DID_EXPRESSION = /^did:(key|ethr|nil):.+$/;
```

This ensures that only valid and supported DID formats pass validation. If future DID methods are added, they can be incorporated explicitly into the regex to preserve validation integrity. Additionally, consider adding unit tests that confirm invalid DIDs (e.g., did:foo:1234) are correctly rejected.

NIL-22 Github Actions Job Timeout

Informational · Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Description: Jobs in GitHub actions should run and potentially fail quickly. Setting an expected time-limit can be useful to ensure performance goals.

Affected Files:
- `./.github/workflows/cd-image.yaml`
- `./.github/workflows/ci.yaml`

Recommendation: For every job inside the `job` section of a workflow, ensure that the key `timeout-minutes` is set.

NIL-23 Lack of Exception Handling for Potentially Throwing Functions

Informational · Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Description: Certain functions within the codebase call methods that can raise exceptions but do not implement proper error handling. For example, in nuc-ts-main/src/core/signer.ts at line 140, the function invokes JSON.parse() without wrapping it in a try/catch block. If the input data is malformed or unexpected, JSON.parse() will throw a runtime exception, which can cause the process to terminate unexpectedly or result in undefined behavior downstream.

This pattern makes the system less resilient to malformed input or data integrity issues. In libraries such as this one, where inputs may be influenced by external sources (e.g., user-provided or network-transmitted data), unhandled exceptions could lead to application crashes, denial of service, or inconsistent state.

Recommendation: Introduce explicit exception handling for all functions that may throw errors, particularly those processing untrusted or dynamic input.

Apply similar error-handling patterns consistently across the codebase. In critical paths, consider defining a structured error type or logging mechanism to ensure failures are captured, diagnosed, and safely propagated without compromising application stability.

Auditor Suggestions

S1 Empty or Incomplete Docstrings in `collections.mapper.ts`

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Description: In nildb-main/src/collections/collections.mapper.ts, beginning around line 71, several functions contain empty or placeholder docstrings (e.g., /** */). These incomplete documentation comments provide no useful information about the function's purpose, parameters, or return values. While the lack of content does not directly impact functionality, it undermines code readability and maintainability. Other developers may misinterpret or overlook the intended behavior of these functions, particularly when the codebase grows or when onboarding new contributors. Inconsistent documentation also reduces the effectiveness of IDE-assisted insights and automated documentation generation.

Recommendation: Ensure all docstrings accurately describe the function's behavior, input parameters, return types, and potential side effects. Additionally, consider adding linting rules or pre-commit checks (e.g., using ESLint with eslint-plugin-jsdoc) to flag empty or malformed docstrings in future commits. This helps maintain consistent and meaningful documentation quality across the codebase.

S2 Identical Value for Conditional Expression

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Description: `nuc-ts-main/src/nuc/policy.ts`, line 308:

```
const selectorValue = typeof selector === "string" ? selector : selector;
```

What is the point of this conditional expression?

Recommendation: Fix the underlying logic to fulfill the purpose of the function.

S3 We Encourage the Fixing of AWS specific, nildb unrelated issues. Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Will address in due course

Description: We found the following items in the organization's AWS configurations, and we encourage the fixing of these items as well.

- Set encryption for AWS SNS topics
- Enforce HTTPs for CloudFront
- Enforce Security Headers (see e.g. the OWASP Secure Headers project) to be served through CloudFront
- Enable Web Applicatoin Firewalls for both CloudFront and API Gateways
- Drop invalid headers before forwarding in the AWS load balancer
- Deletion protection for load balancers
- Protect EC2 instances from deletion, and ensure backups are set up.

Recommendation: We encourage the team to review the aforementioned resources and remediate each item accordingly.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/NillionNetwork/nildb`

- `f64...8a9 ./packages/api/src/app.ts`
- `13b...983 ./packages/api/src/builders/builders.controllers.ts`
- `33c...987 ./packages/api/src/builders/builders.mapper.ts`
- `017...261 ./packages/api/src/builders/builders.repository.ts`
- `4f6...d09 ./packages/api/src/builders/builders.router.ts`
- `7e2...a1a ./packages/api/src/builders/builders.services.ts`
- `6ab...b1c ./packages/api/src/builders/builders.types.ts`
- `f83...517 ./packages/api/src/collections/collections.controllers.ts`
- `f13...e07 ./packages/api/src/collections/collections.mapper.ts`
- `000...63d ./packages/api/src/collections/collections.repository.ts`
- `d68...9f3 ./packages/api/src/collections/collections.router.ts`
- `d27...4d7 ./packages/api/src/collections/collections.services.ts`
- `78a...5f0 ./packages/api/src/collections/collections.types.ts`
- `6bc...c32 ./packages/api/src/common/acl.ts`
- `998...6d7 ./packages/api/src/common/buildinfo.ts`
- `d86...197 ./packages/api/src/common/coercion.ts`
- `155...c6a ./packages/api/src/common/errors.ts`
- `cf8...be9 ./packages/api/src/common/handler.ts`
- `628...bf3 ./packages/api/src/common/logger.ts`
- `f0e...633 ./packages/api/src/common/mongo.ts`
- `58f...967 ./packages/api/src/common/openapi.ts`
- `352...abc ./packages/api/src/common/otel.ts`
- `1b1...bf0 ./packages/api/src/common/shares.ts`
- `9f3...59f ./packages/api/src/common/types.ts`
- `ca3...a79 ./packages/api/src/common/validator.ts`
- `721...d39 ./packages/api/src/data/data.controllers.ts`
- `0e5...0c5 ./packages/api/src/data/data.mapper.ts`
- `5cf...b88 ./packages/api/src/data/data.repository.ts`
- `90c...2d8 ./packages/api/src/data/data.router.ts`
- `c66...07e ./packages/api/src/data/data.services.ts`
- `606...486 ./packages/api/src/data/data.types.ts`
- `7ad...e2f ./packages/api/src/env.ts`
- `396...350 ./packages/api/src/main.ts`
- `6ba...349 ./packages/api/src/middleware/bindings.middleware.ts`
- `3ba...449 ./packages/api/src/middleware/capability.middleware.ts`
- `062...98c ./packages/api/src/middleware/cors.middleware.ts`
- `6ff...e10 ./packages/api/src/middleware/limit-body.middleware.ts`
- `072...1ba ./packages/api/src/middleware/logger.middleware.ts`
- `714...e1c ./packages/api/src/middleware/maintenance.middleware.ts`
- `c86...3c3 ./packages/api/src/middleware/metrics.middleware.ts`
- `feb...1bf ./packages/api/src/middleware/rate-limit.middleware.ts`
- `edd...bdc ./packages/api/src/queries/mongodb_pipeline.json`
- `5f5...0f8 ./packages/api/src/queries/queries.controllers.ts`
- `f4a...434 ./packages/api/src/queries/queries.jobs.repository.ts`
- `2c4...c2f ./packages/api/src/queries/queries.mapper.ts`
- `2f4...e4f ./packages/api/src/queries/queries.repository.ts`
- `b4a...84f ./packages/api/src/queries/queries.router.ts`
- `535...067 ./packages/api/src/queries/queries.services.ts`

- `891...8c4 ./packages/api/src/queries/queries.types.ts`
- `10d...347 ./packages/api/src/system/system.controllers.ts`
- `ea5...887 ./packages/api/src/system/system.mapper.ts`
- `d98...1ef ./packages/api/src/system/system.repository.ts`
- `03e...1ac ./packages/api/src/system/system.router.ts`
- `944...2a0 ./packages/api/src/system/system.services.ts`
- `767...1e2 ./packages/api/src/system/system.types.ts`
- `b77...760 ./packages/api/src/users/users.controllers.ts`
- `7d8...7a8 ./packages/api/src/users/users.mapper.ts`
- `e6f...e13 ./packages/api/src/users/users.repository.ts`
- `cc6...621 ./packages/api/src/users/users.router.ts`
- `a9a...e1f ./packages/api/src/users/users.services.ts`
- `235...96e ./packages/api/src/users/users.types.ts`

Repo: `https://github.com/NillionNetwork/nildb`

- `040...ea5 ./src/app.ts`
- `366...1b1 ./src/builders/builders.controllers.ts`
- `8e7...0f8 ./src/builders/builders.dto.ts`
- `7eb...da5 ./src/builders/builders.mapper.ts`
- `be4...95f ./src/builders/builders.repository.ts`
- `51c...3aa ./src/builders/builders.router.ts`
- `b39...3ac ./src/builders/builders.services.ts`
- `996...c95 ./src/builders/builders.types.ts`
- `bf5...027 ./src/collections/collections.controllers.ts`
- `313...006 ./src/collections/collections.dto.ts`
- `89b...047 ./src/collections/collections.mapper.ts`
- `534...c65 ./src/collections/collections.repository.ts`
- `27c...d3d ./src/collections/collections.router.ts`
- `fb4...047 ./src/collections/collections.services.ts`
- `2d1...5dc ./src/collections/collections.types.ts`
- `22e...730 ./src/common/acl.ts`
- `7b1...4e3 ./src/common/cache.ts`
- `fa6...00b ./src/common/coercion.ts`
- `b5b...954 ./src/common/date.ts`
- `165...273 ./src/common/errors.ts`
- `ac3...787 ./src/common/handler.ts`
- `54b...fb0 ./src/common/logger.ts`
- `b17...af6 ./src/common/mongo.ts`
- `23c...626 ./src/common/nuc-cmd-tree.ts`
- `bff...515 ./src/common/openapi.ts`
- `dd3...422 ./src/common/pagination.dto.ts`
- `00a...cbf ./src/common/paths.ts`
- `1b1...bf0 ./src/common/shares.ts`
- `d50...6a2 ./src/common/types.ts`
- `7d5...a3e ./src/common/validator.ts`
- `577...4ba ./src/data/data.controllers.ts`
- `62b...a68 ./src/data/data.dto.ts`
- `9a7...c70 ./src/data/data.mapper.ts`
- `880...493 ./src/data/data.repository.ts`
- `6db...4b7 ./src/data/data.router.ts`
- `a2f...6fd ./src/data/data.services.ts`
- `888...91b ./src/data/data.types.ts`
- `709...3b3 ./src/env.ts`
- `367...055 ./src/main.ts`

- `059...285 ./src/middleware/bindings.middleware.ts`
- `c3f...670 ./src/middleware/capability.middleware.ts`
- `4a5...875 ./src/middleware/cors.middleware.ts`
- `6d2...614 ./src/middleware/limit-body.middleware.ts`
- `a5a...520 ./src/middleware/logger.middleware.ts`
- `6cf...866 ./src/middleware/maintenance.middleware.ts`
- `837...617 ./src/middleware/rate-limit.middleware.ts`
- `edd...bdc ./src/queries/mongodb_pipeline.json`
- `0c9...766 ./src/queries/queries.controllers.ts`
- `e59...f69 ./src/queries/queries.dto.ts`
- `2b8...e42 ./src/queries/queries.jobs.repository.ts`
- `bb8...a8f ./src/queries/queries.mapper.ts`
- `fa2...22d ./src/queries/queries.repository.ts`
- `f26...cb4 ./src/queries/queries.router.ts`
- `011...4cc ./src/queries/queries.services.ts`
- `690...e08 ./src/queries/queries.types.ts`
- `b19...e10 ./src/system/system.controllers.ts`
- `0fa...1e2 ./src/system/system.dto.ts`
- `ee1...37f ./src/system/system.mapper.ts`
- `96e...2b0 ./src/system/system.repository.ts`
- `b8a...b2f ./src/system/system.router.ts`
- `681...429 ./src/system/system.services.ts`
- `767...1e2 ./src/system/system.types.ts`
- `7f3...b6f ./src/users/users.controllers.ts`
- `811...b10 ./src/users/users.dto.ts`
- `71e...7cc ./src/users/users.mapper.ts`
- `50c...1e4 ./src/users/users.repository.ts`
- `117...5cc ./src/users/users.router.ts`
- `c5d...17f ./src/users/users.services.ts`
- `3b3...46d ./src/users/users.types.ts`

Repo: `https://github.com/NillionNetwork/nildb`

- `040...ea5 ./src/app.ts`
- `366...1b1 ./src/builders/builders.controllers.ts`
- `8e7...0f8 ./src/builders/builders.dto.ts`
- `7eb...da5 ./src/builders/builders.mapper.ts`
- `be4...95f ./src/builders/builders.repository.ts`
- `51c...3aa ./src/builders/builders.router.ts`
- `b39...3ac ./src/builders/builders.services.ts`
- `996...c95 ./src/builders/builders.types.ts`
- `bf5...027 ./src/collections/collections.controllers.ts`
- `313...006 ./src/collections/collections.dto.ts`
- `89b...047 ./src/collections/collections.mapper.ts`
- `534...c65 ./src/collections/collections.repository.ts`
- `27c...d3d ./src/collections/collections.router.ts`
- `fb4...047 ./src/collections/collections.services.ts`
- `2d1...5dc ./src/collections/collections.types.ts`
- `22e...730 ./src/common/acl.ts`
- `7b1...4e3 ./src/common/cache.ts`
- `fa6...00b ./src/common/coercion.ts`
- `b5b...954 ./src/common/date.ts`
- `165...273 ./src/common/errors.ts`
- `ac3...787 ./src/common/handler.ts`
- `54b...fb0 ./src/common/logger.ts`

- `b17...af6 ./src/common/mongo.ts`
- `23c...626 ./src/common/nuc-cmd-tree.ts`
- `bff...515 ./src/common/openapi.ts`
- `794...9cd ./src/common/pagination.dto.ts`
- `00a...cbf ./src/common/paths.ts`
- `1b1...bf0 ./src/common/shares.ts`
- `d50...6a2 ./src/common/types.ts`
- `7d5...a3e ./src/common/validator.ts`
- `577...4ba ./src/data/data.controllers.ts`
- `62b...a68 ./src/data/data.dto.ts`
- `9a7...c70 ./src/data/data.mapper.ts`
- `880...493 ./src/data/data.repository.ts`
- `6db...4b7 ./src/data/data.router.ts`
- `a2f...6fd ./src/data/data.services.ts`
- `888...91b ./src/data/data.types.ts`
- `b3d...0e8 ./src/env.ts`
- `367...055 ./src/main.ts`
- `059...285 ./src/middleware/bindings.middleware.ts`
- `c3f...670 ./src/middleware/capability.middleware.ts`
- `4a5...875 ./src/middleware/cors.middleware.ts`
- `6d2...614 ./src/middleware/limit-body.middleware.ts`
- `a5a...520 ./src/middleware/logger.middleware.ts`
- `6cf...866 ./src/middleware/maintenance.middleware.ts`
- `837...617 ./src/middleware/rate-limit.middleware.ts`
- `edd...bdc ./src/queries/mongodb_pipeline.json`
- `0c9...766 ./src/queries/queries.controllers.ts`
- `e59...f69 ./src/queries/queries.dto.ts`
- `2b8...e42 ./src/queries/queries.jobs.repository.ts`
- `bb8...a8f ./src/queries/queries.mapper.ts`
- `fa2...22d ./src/queries/queries.repository.ts`
- `f26...cb4 ./src/queries/queries.router.ts`
- `011...4cc ./src/queries/queries.services.ts`
- `690...e08 ./src/queries/queries.types.ts`
- `b19...e10 ./src/system/system.controllers.ts`
- `0fa...1e2 ./src/system/system.dto.ts`
- `ee1...37f ./src/system/system.mapper.ts`
- `96e...2b0 ./src/system/system.repository.ts`
- `b8a...b2f ./src/system/system.router.ts`
- `681...429 ./src/system/system.services.ts`
- `767...1e2 ./src/system/system.types.ts`
- `7f3...b6f ./src/users/users.controllers.ts`
- `811...b10 ./src/users/users.dto.ts`
- `71e...7cc ./src/users/users.mapper.ts`
- `50c...1e4 ./src/users/users.repository.ts`
- `117...5cc ./src/users/users.router.ts`
- `c5d...17f ./src/users/users.services.ts`
- `3b3...46d ./src/users/users.types.ts`

Changelog

- 2025-10-29 - Initial report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1100 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republicc, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

© 2026 – Quantstamp, Inc.